MAT61-3

Math for ML

Kernels, SVM, Regularization, Reinforcement Learning, and Attention

---

The mathematical foundations of classical machine learning, extended to the Transformer attention mechanism through the kernel analogy.

Albert School — 30 hours

Contents

1 Kernel functions

A kernel replaces an explicit feature map $\varphi : \mathcal{X} \to \mathcal{F}$ by the inner product it induces, allowing a learning algorithm to operate in $\mathcal{F}$ without ever computing $\varphi(x)$.

Definition 1 (Kernel, Gram matrix) A function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a **kernel** if there is a map $\varphi : \mathcal{X} \to \mathcal{F}$ into an inner-product space with

$$k(x, x') = \langle \varphi(x), \varphi(x') \rangle \quad \text{for all } x, x'.$$

Given points $x_1, ..., x_m$, the **Gram matrix** is $K \in \mathbb{R}^{m \times m}$ with $K_{ij} = k(x_i, x_j)$.

Definition 2 (Positive semi-definite kernel) k is **positive semi-definite (PSD)** if for every finite set of points the Gram matrix K is symmetric and satisfies

$$c^\top K c \geq 0 \quad \text{for all } c \in \mathbb{R}^m.$$

Theorem 3 (Mercer's condition) A symmetric function k is a valid kernel — i.e. it arises from some feature map φ — if and only if it is positive semi-definite. Verifying validity of a proposed k therefore reduces to checking that every Gram matrix it produces is PSD.

Definition 4 (Standard kernels) For $x, x' \in \mathbb{R}^d$:

- **Linear**: $k(x, x') = x^\top x'$.
- **Polynomial**: $k(x, x') = (x^\top x' + c)^p$, with $c \geq 0$ and degree $p \in \mathbb{N}$.
- **Radial basis function (RBF / Gaussian)**: $k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$, with bandwidth $\sigma > 0$.

Definition 5 (Kernel trick) Any algorithm that uses the training data only through inner products $\langle \varphi(x_i), \varphi(x_j) \rangle$ may replace each such inner product by $k(x_i, x_j)$. The computation is then carried out entirely with the Gram matrix K, without forming $\varphi(x)$ — which may be high- or infinite-dimensional (as for the RBF kernel).

Hyperparameter effects. For the polynomial kernel, the degree p controls the order of feature interactions, and larger p yields a more flexible boundary. For the RBF kernel, the bandwidth σ sets the scale of similarity: small σ makes $k(x, x')$ decay quickly with distance (each point influences only its near neighbours, increasing flexibility), while large σ makes the kernel nearly constant (smoother, less flexible).

2 Support Vector Machines

Binary classification with labels $y_i \in \{-1, +1\}$ and a linear decision function $f(x) = w^\top x + b$, classifying by $\text{sign}(f(x))$.

Definition 6 (Margin, hard-margin SVM) For a separating hyperplane the **margin** is the distance from the hyperplane to the nearest training point. The maximum-margin classifier for linearly separable data solves the **hard-margin** problem

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1, \quad i = 1, ..., m.$$

Maximizing the margin is equivalent to minimizing $\|w\|$ under these constraints.

Definition 7 (Soft-margin SVM) When the data are not separable, **slack variables** $\xi_i \geq 0$ permit violations and a penalty parameter $C > 0$ controls their cost:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Large C penalizes violations heavily (narrow margin, fewer errors); small C tolerates violations in exchange for a wider margin.

Definition 8 (Dual formulation) Introducing multipliers $\alpha_i \geq 0$ for the margin constraints, the soft-margin SVM has the dual

$$\max_{\alpha} \sum_{i=1}^{m} \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \, x_i^\top x_j$$

subject to $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$. The optimal weight is $w = \sum_i \alpha_i y_i \, x_i$, and the points with $\alpha_i > 0$ are the **support vectors**.

The dual depends on the data only through the inner products $x_i^\top x_j$. Replacing $x_i^\top x_j$ by a kernel $k(x_i, x_j)$ yields the **kernel SVM**, whose decision function

$$f(x) = \sum_{i=1}^{m} \alpha_i y_i \, k(x_i, x) + b$$

is linear in feature space and nonlinear in the input space. This is why the dual — not the primal — is the formulation that enables kernelization.

3 Kernel PCA

Linear PCA (MAT41-1) diagonalizes the covariance matrix in input space. Kernel PCA performs the same operation in the feature space $\mathcal{F}$ induced by a kernel, accessed only through the Gram matrix.

Definition 9 (Centered Gram matrix) Let K be the Gram matrix $K_{ij} = k(x_i, x_j)$ and let $\mathbf{1}_m$ be the $m \times m$ matrix with every entry $\frac{1}{m}$. The **centered** Gram matrix, corresponding to features centered to zero mean in $\mathcal{F}$, is

$$\tilde{K} = K - \mathbf{1}_m K - K\mathbf{1}_m + \mathbf{1}_m K \mathbf{1}_m.$$

Definition 10 (Kernel PCA) Solve the eigenvalue problem

$$\tilde{K}\,\alpha^{(l)} = m\,\lambda_l\,\alpha^{(l)},$$

and normalize each eigenvector so that $\lambda_l \left\|\alpha^{(l)}\right\|^2 = 1$. The projection of a point x onto the l-th principal component in feature space is

$$\sum_{i=1}^{m} \alpha_i^{(l)}\, k(x_i, x).$$

Comparison with linear PCA. With the linear kernel $k(x, x') = x^\top x'$, kernel PCA reproduces ordinary PCA; a nonlinear kernel extracts components that are nonlinear in the input variables.

Computational tradeoffs. Linear PCA diagonalizes a $d \times d$ covariance matrix at cost $O(d^3)$; kernel PCA diagonalizes the $m \times m$ Gram matrix at cost $O(m^3)$. Kernel PCA is therefore preferable when $m < d$ (or when the feature space is infinite-dimensional) and linear PCA when $d < m$.

4 Reinforcement Learning

This chapter builds on the single Markov-decision example of MAT31-2 (Markov property, state value $V(s)$, and the Bellman equation) and treats the algorithms operationally.

Definition 11 (Markov decision process) An MDP is given by states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transition probabilities $P(s' \mid s, a)$, reward $R(s, a)$, and discount factor $\gamma \in [0, 1)$. A policy π maps states to actions. The Bellman optimality equation for the optimal value V^* is

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a)\, V^*(s') \right].$$

Definition 12 (Value iteration) Initialize V_0 arbitrarily and repeat the update

$$V_{k+1}(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a)\, V_{k(s')} \right]$$

for all s until convergence. The greedy policy with respect to the converged V^* is optimal.

Definition 13 (Policy iteration) Alternate two steps until the policy stops changing:

- **Policy evaluation**: solve $V^{\pi}(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, V^{\pi}(s')$ for the current policy π.
- **Policy improvement**: set $\pi(s) \leftarrow \arg\max_a \Big[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a)\, V^{\pi}(s')\Big]$.

Convergence intuition. Both iterations apply the Bellman operator, which contracts distances between value functions by the factor $\gamma < 1$; repeated application therefore drives any starting estimate toward the unique fixed point V^*.

Control without a model. When P and R are unknown, values are estimated from sampled transitions. **Bootstrapping** updates an estimate using other current estimates rather than waiting for a final return.

Definition 14 (Q-learning update) For the action-value function $Q(s,a)$, on observing a transition (s, a, r, s') apply

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s', a') - Q(s,a)\Big],$$

where $\alpha \in (0, 1]$ is the learning rate.

Definition 15 (ε-greedy exploration) With probability $1 - \varepsilon$ select the greedy action $\arg\max_a Q(s,a)$ (exploitation), and with probability ε select a uniformly random action (exploration).

Q-learning is **off-policy**: the update uses $\max_{a'} Q(s', a')$, the value of the greedy target policy, while the transitions may be generated by a different (e.g. ε-greedy) behaviour policy.

5 Bias-variance framework

For a target $y = f(x) + \varepsilon$ with noise of variance σ^2, and an estimate $\hat{f}$ trained on a random sample, the expected prediction error at a point x decomposes as follows.

Theorem 16 (Bias-variance decomposition)

$$\mathbb{E}\Big[\big(y - \hat{f}(x)\big)^2\Big] = \underbrace{\big(\mathbb{E}\big[\hat{f}(x)\big] - f(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\Big[\big(\hat{f}(x) - \mathbb{E}\big[\hat{f}(x)\big]\big)^2\Big]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

A model that is too simple has high bias and **underfits**: both training and test error are high. A model that is too flexible has high variance and **overfits**: training error is low while test error is high. The gap between training error and test error is the operational signature of variance, and model selection seeks the complexity that minimizes test error.

6 L2 regularization (ridge)

Ridge regression adds a squared-norm penalty to the least-squares objective.

Definition 17 (Ridge regression) For design matrix $X \in \mathbb{R}^{m \times d}$, target y, and $\lambda \geq 0$,

$$\hat{w}_{\text{ridge}} = \arg\min_w \|Xw - y\|^2 + \lambda\|w\|^2 = (X^\top X + \lambda I)^{-1} X^\top y.$$

Because $X^\top X + \lambda I$ is invertible for any $\lambda > 0$, the solution is closed-form and always well-defined, even when $X^\top X$ is singular.

Effect on eigenvalues and shrinkage. If $X^\top X$ has eigenvalues $d_1, ..., d_d$, then $X^\top X + \lambda I$ has eigenvalues $d_j + \lambda$. In the eigenbasis, ridge multiplies the least-squares coefficient along direction j by the factor $\frac{d_j}{d_j+\lambda} < 1$. Directions of small variance (small d_j) are shrunk most; ridge therefore shrinks all coefficients toward zero without setting any exactly to zero.

7 L1 regularization (lasso)

Lasso replaces the squared penalty by the ℓ_1 norm.

Definition 18 (Lasso regression)

$$\hat{w}_{\text{lasso}} = \arg\min_w \|Xw - y\|^2 + \lambda\|w\|_1, \quad \|w\|_1 = \sum_{j=1}^{d} |w_j|.$$

The ℓ_1 ball $\{w : \|w\|_1 \le t\}$ is a diamond (cross-polytope) whose vertices lie on the coordinate axes. The least-squares level sets first meet this constraint region, in general, at a vertex, where some coordinates are exactly zero. Lasso therefore produces **sparse** solutions and performs **feature selection**. Because the ℓ_1 penalty is not differentiable at zero, the lasso has **no closed-form solution** and is solved numerically (e.g. by the proximal / soft-thresholding methods of MAT61-1).

8 L1 versus L2

Definition 19 (Geometric comparison) The constraint region is a diamond (L1) or a ball (L2). The diamond has corners on the axes, so the optimum tends to land on an axis (a zero coefficient); the ball is smooth, so the optimum generically has all coordinates nonzero.

Operationally: L2 (ridge) shrinks coefficients smoothly and keeps all features, is suited to correlated predictors, and has a closed form. L1 (lasso) zeroes out coefficients and selects a subset of features, which is preferred when a sparse or interpretable model is sought or when many features are believed irrelevant.

9 Cross-validation

Cross-validation estimates test error to choose hyperparameters such as λ, C, or σ.

Definition 20 (k-fold cross-validation) Partition the data into k equal folds. For each fold i, train on the other $k-1$ folds and evaluate on fold i, obtaining error e_i. The cross-validation estimate is $\frac{1}{k}\sum_{i=1}^{k} e_i$.

Definition 21 (Leave-one-out) The special case $k = m$: each fold is a single observation. It is nearly unbiased for the test error but costs m model fits.

Model selection procedure. For each candidate hyperparameter value, compute the cross-validation error; select the value minimizing it; refit the model on the full training set at that value; and report performance on a separate held-out test set. The procedure trades computational cost (number of fits) against the variance of the error estimate.

10 Transformer attention as a kernel operation

Attention computes, for each query, a weighted average of values, with weights given by a similarity kernel between the query and the keys in a learned embedding space.

Definition 22 (Scaled dot-product attention) For queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{n \times d_k}$, and values $V \in \mathbb{R}^{n \times d_v}$,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The entry $(QK^\top)_{ij} = q_i^\top k_j$ is the dot-product similarity between query i and key j, scaled by $\frac{1}{\sqrt{d_k}}$.

The scaled dot product $q_i^\top k_j/\sqrt{d_k}$ plays the role of a similarity kernel between query and key in the learned embedding space. The softmax over j turns these similarities into nonnegative weights summing to one,

$$\alpha_{ij} = \frac{\exp\left(q_i^\top k_j/\sqrt{d_k}\right)}{\sum_{j'} \exp\left(q_i^\top k_{j'}/\sqrt{d_k}\right)},$$

which is a **Boltzmann (Gibbs) distribution** over the keys, with the scaled similarity as negative energy and temperature $\sqrt{d_k}$. The output for query i is $\sum_j \alpha_{ij}\, v_j$.

Definition 23 (Multi-head attention) Project Q, K, V through h learned linear maps into h subspaces, apply scaled dot-product attention in each, and concatenate:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \quad \text{head}_l = \text{Attention}\left(QW_l^Q, KW_l^K, VW_l^V\right).$$

Each head is a parallel learned projection, computing attention under a different learned similarity.

11 The Transformer block

This chapter composes attention with components reviewed from MAT41-3 (residual connections and layer normalization — review, not new content).

Definition 24 (Residual connection) A sublayer f is wrapped as $x + f(x)$: the identity path carries the input forward and f adds a perturbation, which improves gradient flow through deep stacks.

Definition 25 (Layer normalization) For an activation vector x with mean μ and variance σ^2 over its components,

$$\mathrm{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta,$$

with learned scale γ and shift β. It standardizes each activation vector to stabilize training.

Definition 26 (Transformer block) One block composes a multi-head self-attention sublayer and a position-wise feed-forward sublayer, each wrapped in a residual connection and layer normalization:

$$z = \mathrm{LayerNorm}(x + \mathrm{MultiHead}(x, x, x)),$$
$$y = \mathrm{LayerNorm}(z + \mathrm{FFN}(z)),$$

where $\mathrm{FFN}(z) = \sigma(zW_1 + b_1)W_2 + b_2$ is a two-layer feed-forward map.

A Transformer is the composition of many such blocks. As a mathematical object, each block is the composition of: a kernel-like attention mixing across positions, an identity (residual) pathway, a normalization map, and a per-position feed-forward transformation.